classmate
Date
Page

ప్రోగ్రామ్.

Introduction to Python :-

* General purpose programming language which means it is used for development, testing and deployment of Software applications
* High level programming language means Human readable and Easily Understandable language.
* Python having extensive ecosystem of libraries and frameworks
    1. Data Handling – Numpy
    2. Data Visualization – Matplotlib, Seaborn
    3. ML algorithm implementation – Scikit-Learn
    4. Deep learning – Tensorflow, Keras, Pytorch
* Python Supports both functional and object oriented approach
* Python is an Interpreted programming language.
* Python was Created by Guido Van Rossum in 1991
* It is Having many applications like Standalone of
    1. Standalone applications
    2. Web applications
    3. Database applications
    4. Bigdata applications
    5. Machine learning applications

Keywords :-

* The words which is reserved to do specific purpose is called Keyw
* We cannot use keywords as Identifier's
* Keywords are case sensitive [True, False, None → only Capital alphabet
* There are 35 keywords present in 3.13.1 version of Python

Flow control	Variable	Funcion/Method	Object oriented	Module	Logical	Checking	Deletion
If	Global	def	Class	import	and	is	del
else	Nonlocal	with	Exception-Handl	from	or	in	
elif	Built-in-Cons	lambda	try	as	not		Asynchor
for	True	yield	except				async
while	False	return	finally				await
break							
continue							
pass							

Scanned with OKEN Scanner

Ex: import keyword
print(keyword.kwlist) # Prints 35 keywords as list
print(len(keyword.kwlist)) # O/p - 35

Ways to compile Python program:
Edit+, Notepad, Notepad++, VS Code, Anaconda; Jupyter notebook, Google colab, Spyder, Atom, Idle (Integrated development Environ

print() → predefined function
* Display the output

Python program flow:

Source code → Byte code → Python → Output
Virtual Machine

Source code: The written python code, saved with .py (or) .Python exten
Byte Code: While running the program, Interpreter converts source cod into Byte code file (Intermediate file)
PVM: Byte code file cannot understandable by microprocessor to generate the output, then here PVM converts Byte c into Machine understandable format, then finally we get output

Identifiers:

* The name in a python program is called Identifier.
* The name can be
    1. Package name.
    2. Module name
    3. Variable name
    4. Function name
    5. Class name
    6. Method name
* Rules for Identifier's
    1. Identifiers should by
        → Either lower case or upper case
        → Use 0-9 numbers only
        → If there is only multiple Combine those only with "_"
    2. Identifiers are Case Sensitive
    3. Spaces are not allowed in between the Identifier's
    4. We cannot use Keywords as Identifier's
    5. Identifier's Should not starts with digits

Example:

trick = 20 → print(Trick) → Name error
trick A = 10 → Syntax error
True = 20 → Syntax error.
1abc = 20 → Syntax error.

Comments :-

Single line Comments → # This is Naveen
Multi line Comments = # I'm here to help you
Don't feel bad.

Doc String → """ Hey, where are you going
I'm here
Can I help you regarding this?"""

Scanned with OKEN Scanner

Variables :-

* Variable is a reserved memory location to store the values
* Purpose of variable is to represent the data
* Data is a collection of facts
  → Facts can be
    1. Alphabets [A-Z]
    2. Symbols [,,&,",$,#,]
    3. Numbers [1-∞]
    4. Alphanumeric [ABC12]

* We should write name of the variable with lower case only if there is any multiple words combine those only with "_"
* Properties of Variable, where each variable contain
  * Name → age = 16
  * Type → int
  * Value → 16
* Creating Multiple variables in a single line.
  a, b, c = 10, 20, 'Naveen'
* Assigning single value to a multiple variables
  a = b = c = 16
* id() → predefined function used to returns the memory storage location of variables which holds values

id()

a = 10

b = 20

id(a) → o/p - 1407 ... 7928

id(b) → o/p - 1407 ... 8248

Data types:

* The type of data present in a given variable or memory location is called Datatype
* type() → predefined function used to check type of data present in a given variable or memory location.

Types of datatypes:

1. Built-in-datatypes:
   1. Numeric datatypes → int → Numbers without decimals
      → float → Numbers with decimals
      → Complex → Combination of [real part + imaginary]
   2. bool datatype → It represents boolean values
      → Boolean datatype having 2 values only
      1. True
      2. False
      → Internally python represents
      * True as 1
      * False as 0
   3. None datatype → It represents object doesn't contain any value
      → If anyobject cannot assign with any value then returns none we can assign that object with 'None'
      → The function/method can return None data type.
   4. Sequence → A group of values stored in a single entity is called Sequence.
      1. String
      2. List
      3. tuple
      4. Set
      5. dictionary
      6. range → A group of values from starting to

Type Casting:

int()
float()
str()
list()
tuple()
set()
dict()

Scanned with OKEN Scanner

Strings:

* A group of characters enclosed in a single quote, double quote or triple quote is called String

ex: single_quoted = 'Hello 123'
double_quoted = " Hello bacci"
triple_quoted = """ Hello siri """

* It can store same or different of values

ex: var = 'Hello, 1234 + 1+2j'

* Duplicate elements are allowed
* Size of String is fixed once we created
* Insertion order is also fixed
* String is having Immutable Nature
* Indexing and Slicing is possible for the String.

ex: a = 'Naveen' → a[1] ⇒ o/p: a, a[2:] → o/p: veen

(* end is not included in slicing)

Note:

Mutable nature: Once we create an object, we can change, modify or update the state of existing object, this behaviour is called Mutable nature.

ex: list, set, dictionary

Immutable nature: Once we create an object, the state of existing object cannot be changed, modified or updated this behaviour is called Immutable nature

ex: int, float, string, tuple

Mathematical Operations:

* For string's Concatenation/joining the multiple strings, we can use '+'

ex: a = 'python'
b = 'programming'
print(a+b) → o/p: 'Python programming'

* For String repetition, we can use '*'

ex: print(a*2) → o/p: Python Python

len() → predefined function used to check no. of characters in string

Membership operators:
* in → in operator returns 'True', if the substring found in main string
Ex: print ('p' in 'python') → o/p: True.
* not in : not in operators returns True, if the substring not found in main string [opposite of in operator]
Ex: print ('s' not in 'python') → o/p True.

dir() → predefined function which returns list of attributes and methods of any class or method.

Important methods in String class:
* Predefined method, we can access this method by using string object only.
1. upper() → Converts the lower case string into upper case.
2. lower() → Converts the upper case string into lower case
3. strip() → Removes left and right hand side spaces of the string
4. split() → Split the string into list of string based on separator
default split mtd splits on space
5. count() → Count the occurrences of string on main string
6. replace() → replace the old string with new string
* replace method never performs changes on existing string, it creates new string object
* We can check this by id() function
Ex: a = 'abcdefgh'
a.replace('b', 'Naveen') o/p: "aNaveencdefgh"

7. find() → method used to find the first occurence of substring from starting index [left - right]
8. rfind() → method searches right - left and returns highest index where substring found.

Scanned with OKEN Scanner

9. Capitalize() → method used to capitalize the first letter of first word in a sentence

ex: a='hello man'

a. capitalize() → o/p: 'Hello man'

10. title() → method used to capitalize the every word in a sentence.

ex: a.title() → o/p → 'Hello Man'

11. Swapcase() → Converts lower case into upper case and upper case into lower case.

ex: a='Hello Buddy'

a.swapcase() → o/p: 'hELLO bUDDY'

12. join() → Combine multiple words into single string

ex:

1. l=['I', 'Love', 'Python']

print(" ".join(l)) → o/p: I Love Python

2. print('-'.join(l)) → o/p: I-Love-Python

3. print(','.join('Hello')) → o/p: 'H','e','l','l','o'

13. format() → predefined mtd in python

→ format method format and string with passed string specified value and insert them inside the string's placeholder

→ String's placeholder can be defined as '{}'

```
l = [10, 20, 30, 40]
for i in l:
    print('My rde number is {}'.format(i))
```

14. endswith() → used to check and returns boolean value

15. Startswith() → used to check whether startswith or not and returns Boolean values.

16. index() → This mtd used to return the index of substring from starting.

Scanned with OKEN Scanner

classmate
Date
Page

List :-

↑ Mutable

An ordered collection of Values Stored in a Single entity is called lis

* We can create list by using list predefined
  1. list() → function
  2. [] → Square brackets
* It can store Same or different type of elements

  Ex: list1 = [1,2,3,4,5]

  list2 ⇒ [1,2,'Naveen', 10.5, 1+2j, (1,2,3)]
* Insertion order is fixed once we create an object/list
* Duplicate elements are allowed → list = [1,2,3,3,2,'Nav','P']
* Size of list can dynamically increases.
* List is having mutable nature.
* Indexing & Slicing is possible [Start : end : step]

  Ex: l = [1,2,3,'Naveen', 'Pokiri', 10.0, (1,2,3,4)]

  # default starts with '0' index and default step size - 1 ⇒ print(l[2]) ⇒ o/p: 3

  and not included the end if we mentioned print(l[2:4]) ⇒ o/p: [3,'Naveen']

  print(l[2::]) ⇒ o/p: [3,'pokiri',(1,2,3,4)]

  print(l[::-1]) ⇒ o/p: [(1,2,3,4), 10.0, 'pokiri', 'Naveen', 3,

  Print(l[50]) → o/p: Indexerror.

Type casting:

* Ex 1) r = range(10) ⇒ o/p: 0,1,2,3,4,5,6,7,8,9

  list(r) ⇒ o/p: [0,1,2,3,4,5,6,7,8,9]

  2) a = 'Naveen'

  print(list(a)) ⇒ o/p: ['N','a','v','e','e','N']

* len() → predefined function used to return no. of elements present in the list.

Mathematical operators:

* Concatenation (+) → By using this we can add multiple lists int single list, Ex: ['N','a','v','e','e','n','N','a','v','e','e
* Repetition operator (*) → used to multiply the list with no. of times and returns the new lis

  Ex: ['N','a','v','e','e','n'

Operations we can do with lists :

* Add the element
* remove the element
* Access elements by using indexing and slicing operations
* Iterating one by one elements by using for loop

Methods in List data structure :-

1. Count ( ) → this mtd used to return no. of occurences of value in a list
2. append ( ) → this mtd used to add the element / value at the end of the list
3. insert(p,v) → this mtd used to add the element at specific location

eg : l = [1,2,3,4,5,6,10.0,'String',1+2j]

l.append(5005) ⇒ # o/p : [1,2,3,4,...,1+2j,5005]

l.insert(2,5000) ⇒ # [1,2,5000,3,4,...,1+2j]

pos value

4. remove ( ) → It can remove the first occurences element / value

If value is not present "value error"

l.remove(3) ⇒ # [1,2,4,5,6...,1+2j]

5. reverse ( ) → It can reverse the list

l.reverse ( ) ⇒ [1+2j,'String',...,3,2,1]

6. Sort ( ) → It can sort the values in order like for numbers → ascending → desc for strings → Alphabetical orde

7. Sorted ( ) → It is a function which is almost have same functionality

l = [1,3,4,5,7] Eg: l.Sort ( ) ⇒ o/p : [1,2,3,4,5]

Sorted (l) ⇒ o/p : [1,2,3,4,5]

Scanned with OKEN Scanner

pop() → This method remove and returns the element at Specified index

* If index is not given, by default it removes last element.
* If index is out of range given → Index error.

Ex:

```
l = [1,2,3,4,5,6]
l.pop(3) => o/p: [1,2,3,5,6]
l.pop() => o/p: [1,2,3,4,5]
l.pop(20) => o/p: Index error
```

Accessing and adding an elements in list:

```
l = [1, 2, 3, '10', '20ab', 10.0, (1,2,3,4), [1, 2, [3,4]], 'str']
```

Accessing:

```
=> l[2]      o/p: 3
=> l[6]      o/p: (1,2,3,4)
=> l[7][2]   o/p: [3,4]
```

Adding: `l[2] = 35` => `[0,1,35,2`

`l[7] = [100,200]` => o/p:

`[1,2,3,..., [1,2,[3,4]], [100,200]]`

List Comphrension:

* List comphrension is the syntax by applying that the return type also a list
* List comphrension represents creating new lists from iterable object like list, set, dictionary, tuple and range.
* List comphrension takes input as iterable, apply conditional logic on every item and returns a new list.
* List comphrension code is very concise coay.

Syntax: [expression for item1 in iterable if statement]

Ex: `[ i for i in range(10) if i%2 == 0]`

o/p: [0,2,4,6,8,10,12,14,16,18]

Scanned with OKEN Scanner

Tuple :

- An ordered collection of immutable elements stored in a single entity within parenthesis ( ) is called tuple.
- We can create tuple by using "tuple()" function or (1,2,3)
- It can store same or different types of elements
- Insertion order is fixed in tuple once created
- Duplicate elements are allowed in tuple
- Once tuple created, size of the tuple is fixed ⇒ o/p: Value error
- We can access elements in tuple by using Indexing & Slicing
- Tuple is having immutable nature.

Ex: mytuple = (1, 2, 10.0, 'String', [1,2,3])
print(dtype(mytuple)) ⇒ tuple

mytuple[0] = 'Naveen' ⇒ o/p: Value error.

unpack tuples: x, y, z = (10, 20, 'Naveen')
o/p: x = 10, y = 20, z = 'Naveen'
o/p: print(x, y, z) ⇒ 10 20 'Naveen'
x = (10) ⇒ dtype(x) ⇒ o/p: int
x = (10,) ⇒ dtype(x) ⇒ o/p: tuple.

Mathematical operators :

- By using '+' operator we can concatenate tuples
tuple1 = (1,2,3) ; tuple2 = (10,20,30)
tuple1 + tuple2 ⇒ (1,2,3,4,5,6)

- By using '*' operator we can do repetition of tuple

tuple1 * 2 ⇒ o/p: (1,2,3,1,2,3)

Scanned with OKEN Scanner

len() → Predefined function used to return no. of elements in tuple.

Methods in tuple :-

1. count() → It is used to count the no. of occourences of string element/any value in the tuple

2. index() → It is used to return the first occourence index of given element
* If it is not found it returns "Valueerror".

* Tuple is immutable but we can add element to list in tuple.

Ex: t = (11, 22, [33, 44], 55, 66)
t[2].append('Naveen')
o/p: (11, 22, [33, 44, 'Naveen'], 55, 66)

Sets :-

Set is an unordered and collection of mutable collection of unique elements stored in a single entity is called set

* We can create Set by using set() or { } → curly braces
* It can store same or different types of elements
* Insertion order is not fixed [Because unordered] ⇒ If try Type error
* Duplicate elements are not allowed
* Indexing and slicing is not possible [Because no insertion order]
* It is having mutable nature.

Methods in Set Class :-

1. add() → By using this method we can add the element to the set (It can add at any index because it doesn't have proper insertion order)

2. remove() → By using this we can remove the element. If element not present, we will get KeyError raised #

3. update() → By using this we can add multiple values to the set

Ex: S = {1,2,3}
S.update({4,5,6}) ⇒ o/p: {1,2,3,4,5,6}

4. discard() → Removes a specified element if it is present # No error raised

Ex: S.discard(10) ⇒ print(S) ⇒ o/p: {1,2,3}

5. pop() → Removes and returns an arbitary element. Raise an error if the set is empty

Removes an element at any position and returns what it removes

S = {1,2,3,4,5}
Ex: S.pop(-) ⇒ o/p: 1 # {2,3,4,5}
S.pop() ⇒ o/p: 3 # {2,4,5}

Scanned with OKEN Scanner

* Union() → Returns all unique elements from both sets (combine everything)

Ex: A = {1,2,3,4}
B = {3,4,5} # use A|B
print(A.union(B)) ⇒ O/P: {1,2,3,4,5,6,7,8}

* intersection() → Returns common elements present in both sets [common elements]

use A&B
Ex: A.intersection(B) ⇒ O/P: {3,4}

* difference(:) → Returns elements that are in the first set but not in the second set

A-B Ex: print(A.difference(B)) ⇒ O/P: {1,2}

*

Dictionary:

A group of objects stored as key-value pairs in a single entity is called dictionary

* We can create dictionary by using {k:v} pair (or) dict()
* ~~Dictionary~~ Keys stores same type of elements
* It can store like keys and values stores same or different types of elements.
* Insertion order is fixed [After 3.7+version]
* Duplicate keys are not allowed but duplicate values are allowed.
* Size of dictionary increases dynamically
* Dictionary is having mutable nature.
* We can access values by using only keys not indexes.

Syntax: d = {key: value1, key2: value2}

d = {'Naveen': 95155, 'Kote': 94902}

print(type(d)) ⇒ o/p: <class 'dict'>

Adding and Accessing elements and Modify elements:

Adding:- d['pencil'] = 1002

d['Sharpner'] = 50023

print(d) ⇒ o/p: {'Naveen': 95155, 'Kote': 94902, 'pencil': 1002, 'Sharpner': 50023}

Accessing:- d['Naveen'] ⇒ o/p: 95155

print(d['pencil'] ⇒ o/p: 1002

print(d['power']) ⇒ o/p: KeyError

We can access elements by using keys

Modify elements by using Keys:

d['Naveen'] = 'Krishna'

print(d) ⇒ o/p: {'Naveen': 'Krishna', 'Kote': 94902}

Dictionary:

A group of objects stored as key-value pairs in a single entity is called dictionary

* We can create dictionary by using {k:v} pair (or) dict()
* ~~Dictionary~~ Keys stores same ~~type of elements~~
* It can store like keys and values stores same or different types of elements.
* Insertion order is fixed [After 3.7+version]
* Duplicate keys are not allowed but duplicate values are allowed.
* Size of dictionary increases dynamically
* Dictionary is having mutable nature.
* We can access values by using only keys not indexes.

Syntax: d = {key: value1, key2: value2}

d = {'Naveen': 95155, 'Kote': 94902}

print(type(d)) => o/p: <class 'dict'>

Adding and Accessing elements and Modify elements:

Adding:- d['pencil'] = 1002

d['Sharpner'] = 50023

print(d) => o/p: {'Naveen': 95155, 'Kote': 94902, 'pencil': 1002, 'Sharpner': 50023}

Accessing:- d['Naveen'] => o/p: 95155

print(d['pencil'] => o/p: 1002

print(d['power']) => o/p: keyerror

* We can access elements by using keys

Modify elements by using keys:

d['Naveen'] = 'Krishna'

print(d) => o/p: {'Naveen': 'Krishna', 'Kote': 94902

Note:
* We cannot use list, dict, dictionary as keys if we use it return "Type error": Unhashable type: 'dict'.
* Nested dictionary

d = {1: 'Naveen', 2: 'Kote', 3: {'A': 'datascientist', 'B': 'pune'}}

* By using dict() → predefined function we can create ~~dictionary~~ Variable

Ex: this_dict = dict(name = 'John', age = 36, country = 'India')
o/p: print(this_dict)
{'name': 'John', 'age': 36, 'country': 'India'}

* We can remove elements by using
1. del keyword ⇒ ex: del d['name'], del d → entire dictionary
2. clear() method.

Methods in dictionary class:-

1. clear() → Remove all key:value pairs in dictionary and keeps empty dictionary.
2. keys() → Returns all keys present in dictionary.
3. values() → Returns all values present in dictionary.
4. items() → We can access keys, values from dictionary.
Ex: for K,V in d.items():
print(K,V)
5. pop() → Removes the key what we are mentioned and returns its value.
Ex: d.pop('Naveen') o/p: 9515.
6. popitem() → Removes the random Key:value pair and returns key,value pair as tuple
Ex: d.popitem() ⇒ ('Kote', 9490)
7. Update() ⇒ We can update the dictionary by using another key:value pair of dictionary.

pop() ⇒ key compulsory
If key is not present throws KeyError.
popitem() → no argument
(key) → randomly removes

Operators :-

* Operators are Special symbols or keywords that performs an operation.
* Operators acts on some variables, values or expressions is called Operands.

Types of Operators :

1. Arithemetic operators :

'+' → addition
'-' → Subtraction
'*' → Multiplication
'/' → Division
'%' → Remainder
'**' → Exponentiation
'//' → Floordivision - division rounded down to the nearest whole number.

2. Comparision Operators :

== → Equal to
!= → Not equal to
< → Less than
> → Greater than
<= → Less than or equal to
>= → Greater than or equal to

3. Logical Operators :

and [Logical AND]
or : [Logical OR]
not [Logical NOT]

4. Assignment operators :

=, +=, -=, /=, %=, **=, //=

+= → a+b=

5. Membership Operators :

in → Returns true, if element found in the collection
Returns False, if element ~~not~~ found in the collecti

not in → Returns True, if element not found
Returns False, if element found :

6. Special operators :

is and is not are identity operators in python.

is
is not } used to check if two values/variable are locat
or not on same part of memory

Flow Control:

The order of statement execution is called Flow control.

Types of execution:

1. Sequential execution
2. Conditional execution
3. Looping execution

Sequential execution:

* If we execute statements from top to bottom which means Sequential
* By using Sequential statement, we can develop only simple progra

Conditional execution:

* Based on conditions, statements can be executed
* Conditional statements are used to develop better & Complex programs.

Looping execution:

* Statements can executed randomly and repeatedly based on the conditions

Conditional Statements:

* if is a keyword in python
* if statement contains expression or condition
* As per syntax : is mandatory otherwise it throws Syntax error.
* After if statement we need to follow proper indentation otherwise it throws indentation error.
* Condition gives the result as True or False
* If the condition result is True → if block would executed
* If the condition result is False → if block won't executed

else:
* If the condition result is True → if block statements will executed
* If the condition result is False → else block statements will executed

if else elif :
* If the condition result is True, then any matched if or elif block statements will executed.
* If all if and elif conditions results are False, then else block statements will executes

Looping:
* for is a keyword in python
* for loop is used to iterate elements one by one from sequence like String, list, tuple.. etc.
* While iterating elements from sequence we can perform operation on every element

While loop:
* While is a keyword in python
* If we want to execute a group of statements repeatedly until the condition reaches False, then we should go for while loop

while:
else:

break:
* Break is a keyword in python, we can use break statement inside the loops.
* By using break, we can break the execution based on some condition
* break statement is used to terminate for and while loops

Continue: Continue Statement to skip the current iteration & Continue nex

Pass: Place holder for future code.
* While a null operation, nothing happens

Function:

A group of statements which performs the task is called Function

Advantages:

1. Modularity → Functions breakdown complex processes into smaller and modular pieces
2. Reusability → Once function is defined, we can use multiple times throughout a program
3. Organization → Functions helps to organize code into logical groups
4. Maintenance → If change need, we only need to update the code at one place.

Types of Functions:

1. Predefined functions
2. Userdefined functions.

1. Predefined functions:

The functions which are already existing in python is called predefined functions.

Ex: input(), id(), print(), type() ... etc

2. User defined functions:

The functions which are explicitely created by programmer.

Terminology:

def keyword
name of the function:
paranthesis () and parameters (if required)
colon:
function body
return type.

Doc string describes what function does [""" """]

* A function can contains mainly two parts
    1. Creating a function
    2. Calling a function

Ex: def addition(x,y): # Function Creation
        print(x+y)
    addition(10,20) # Function Calling

* We need to call the function then only function body will be executed.
* Function name during creation and Function call should be matched otherwise it throws Name error
* We can call a function on another function also.

Ex: def m1():
        print('First function')
        m2()
    def m2():
        print('Second function')

    m1() O/P: First function
              Second function

Parameters: These are the variables which are listed inside the parenthesis in function definition
Arguments: These are the actual values passed to the function when it is called

Ex: def add_numbers(a,b):
        result = a+b # a,b → parameters
        print(result) # 10,20 → Arguments
    add_numbers(10,20)

Scanned with OKEN Scanner

Return Statement :

* return is a keyword in python.
* We can use return keyword inside the method or function otherwise we will get an error.
* If a function contains return statement then that function we need to assign to a variable because we can use that variable further in our program

Ex: def add_two(a,b):
    result = a+b
    return result
ab = add_two(25,30)

print(ab) → o/p: 55

* If a function not return anything by default that function returns None datatype

Ex: def m1():
    print("Hello world")
a = m1()
print(m1()) → o/p: None

* A function can return multiple values, while function call if we assign function call to a single variable, it will be stored as tuple

Ex: def m1(a,b):
    a = 10
    b = 20
    return a, b

If
x = m1()
print(x) ⇒ (10,20)

x,y = m1() → o/p: print(x) ⇒ 10
print(y) ⇒ 20

Scope of Variables:

Variable defined within a function are scoped to the function and not accesible outside of it. This is called local scope

Ex:

```
def multiply(x,y):
    result = x*y
    print(result) # 'result' is local to the
    return result  multiply function
print(multiply(10,20))
```

O/p: 6 6

print(result) → O/p: Name error: result not defined

Variable defined outside the function are scoped to accessible anywhere in the program. This is called Global scope.

Function creation ←

```
Ex: global_var = 'This is global variable'   → global variable
    def test_life_time():
        """ This function test the life time of a variable """
        local_var = 'this is local variable'.
        print(local_var)                      → Local variable
        print(global_var)
test_life_time()   O/p: 'This is local variable'
                        'This is global variable'
print(global_var) → This is global variable.
print(local_var) → # Name error
```

Doc string used to document of the function ←

Function calling ←

global Keyword → used to make the variable which is defined inside function also global

Function - 2 :

Recursion: Recursion is a programming technique where a function calls itself to solve smaller instances of the same problem until it reaches a base case that stops further calls.

Ex:

1) def Countdown(n):
    if n==0: # base case
        return
    print(n)
    Countdown(n-1)
Countdown(10) # Recursive call

o/p: 10 9 8 7 6 5 4 3 2 1

2) def factorial(n):
    if n==0: # Base case
        return 1
    return n * factorial(n-1) # Recursive call
a = factorial(5)
print(a) => 125

Disadvantages:

1. Infinite recursion :-
2. Performance :- Less efficient than thier iterative counterp due to overhead of function calls and returns
3. Redundant Calculations :

Lambda functions :-

* Lambda is a keyword in python
* By using lambda keyword we can create lambda function
* Lambda function is also called as Anonymous function
* A function without name is called Anonymous function
* Lambda function will process the input and returns the result

Syntax : lambda argument_list : expression

Advantage :
Quick & concise way to create small functions.

Ex : s = lambda x : x*2
print(s(2)) ⇒ 4

Limitations :
Restricted to single expression.

a = lambda d,f : d+f
print (a(10,20)) ⇒ o/p : 30

map() :

* map(fun, iterable) is a predefined function in python
* This function takes iterable and apply conditional logic on every item and returns the new list

Syntax : map(fun, iterable)

Ex : l = [10,20,30,40]
m = map(lambda x : x+2, l)
print(list(m)) ⇒ [12,22,32,42]

filter(P_1, P_2) :

* This function takes iterable and apply filter logic on every item and returns new iterable.

Ex : f = filter(lambda x : x>22, l)
print(list(f)) → [32,42]

reduce(P_1, P_2) :-

* predefined function existing in functools module
* This function apply function of two arguments cummulatively to the items of sequence from left to right

Ex from functools import reduce

Arguments :

1. Positional arguments :

* Possitional arguments are the arguments passed to the function in a correct positional order.
* No. of arguments and position of arguments should be mat otherwise we will get an error.

Ex: def sub(x,y): # If we change the no. of
print(x-y) arguments we will get an
sub(20,10) error because this func accepts two arguments on

2. Keyword arguments :

* Keyword arguments are the arguments that recognize the parameter by the name of the parameters.
* We can use identifier to provide values to the function parame

Ex:

```
def cart(product, price):
    print("product is", product, "price is price")
cart('product='Bangles', 100)
```

o/p: product is Bangles price is 100

3. Default arguments :

* During function definition we can provide default values to the function parameters.
* While creating a program we can provide values to the function parameters.

Ex: def cart(product, price=40):
print("product is =", product)
print("price is =", price)
~~def k('Bangles')~~
cart(product = "Bangles")

② Ex:
```
def cart(product, price=10000):
    print(product)
    print(price)
cart(product='Mobile', price=50,000)
```
O/P: Mobile
50,000
} If we mention during function call it takes otherwise default value will be considered

* args or Variable length arguments:
* * args also called as variable length arguments.
* Sometimes our requirement can be like we need to provide more values to the function parameter to process the result.
* It represents single star symbol before the argument name
* Internally provided values will be stored in a tuple

Ex:
```
def m(*args):
    print(a)
m(10,20,30)
```
O/P: (10,20,30)

** Kwargs or Keyword Variable length arguments:
* It represents ** (double asterik symbol) before the parameter name
* Internally provided values should be stored in a key and Value pair in dictionary.

Ex:
```
def person_info(**kwargs):
    print(kwargs)
    info=" "
    for key,value in kwargs.items():
        info+=f'{key} is {value}'
    return info
print(person_info(name='Carol', age=20, city='Hyd'))
```
O/P:
name is carol, age is 20. city is Hyd.

Scanned with OKEN Scanner

Example of both *args and **kwargs :

```
def setup_profile(name, email, *args, **kwargs):
    print('name : ' name)
    print('email : ' email)
    print(*args) for i in args:
        print(i)
    for i, j in kwargs.items():
        print(f'{i} is {j}')

setup_profile('Naveen', 'Naveen@gmail.com'
    , 'Loves Python', 'Write code',
    Age=28, occupation='Developer,
    Hobby='cycling')
```

O/P:

```
name : Naveen
email : Naveen@gmail.com
Loves Python
Write code
Occupation is Developer
Hobby is Cycling
```

Scanned with OKEN Scanner

Unpacking arguments :

Ex: def person_info(name, age, city):
    return f"{name} is {age} years old and lives in {city}."

Normal ← print(person_info('Balaji', 28, 'Hyd'))

o/p: Balaji is 28 years old and lives in Hyd

unpacking a list into positional arguments → details = ['Naveen', 24, 'Hyd']
print(personinfo(*details))

o/p: Naveen is 24 years old and lives in Hyd

unpacking a dictionary into keyword arguments → details = {'name': 'Naveen', age: 24, 'city': 'Hyd'}
print(personinfo(**details))

Real life example :

Logging is extensively used in many applications & advantages are

1. Error Tracking :
2. Performance monitoring
3. Status check ... etc

```
import datetime
def log_message(*messages, **additional_data):
    """Logs with a optional additional data"""
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    full_message = " | ".join(messages)
    logentry = f"{timestamp} - Message: {full_message}"
    for key, value in additional_data.items():
        logentry += f", {key.upper()}: {value}"
    print(logentry)
log_message('Application started', 'No errors', urgency='low', user_id='1234')
```

o/p: 2026-10-22 15:51:10 Application started | No errors. [illegible] USER_ID: 123

Module:

* A Saved python file is called Module
* This file contains group of classes, methods, functions and Variables.
* Any file .py or .python extension is recognized as module.

import ():

* import is a keyword in python
* By using import keyword we can import the modules in our progr
* Once we imported the module, we can use members of module

Eg: import datetime.

from and import keywords:

* from is a keyword in python
* By using from keyword we can import specific members of the module.
* Advantage of from keyword is we can directly access mem directly without using module name.

Eg: from sklearn.preprocessing import train_test_spli

Aliasing (as keyword):

* By using as keyword we can alias the members of a module.

Eg:

```
from sklearn.preprocessing import train_test_split as tst
```

Package:

* Any folder which contains group of modules
* Any folder which contains __init__.py file then it is considered as package.
* __init__.py can be empty also
* The package contains subpackages also.

Advantages:

1. Resolve naming conflicts
2. Identify our components uniquely.
3. Improves modularity of the application.

Exception Handling:

In any programming language there are 2 types of errors.

1. Syntax errors.
2. Runtime errors.

Syntax errors:

The errors which occours due to invalid syntax is called Syntax errors

Runtime errors:

While executing the program if something goes wrong, we will get Runtime errors. Those might be cause of

* End user input
* Programming language
* Memory problems

Exception:

* An unwanted, unexpected event which disturbs the normal flow of execution is called exception
* When an exception occours then immediately program will terminated abnormally.

Runtime errors

Exception errors: The errors which are detected during execution

1. ValueError: Correct datatype but wrong value ex: int('abc')
2. TypeError: Wrong datatype used. ex: 5+'10'
3. IndexError: Index out of range. ex: l=[5,10,15]; l[5]
4. KeyError: Dictionary key not found. ex: d={'name':'Nani'} d['age']
5. FileNotFoundError: File doesn't exist ex: open('data.txt')
6. ZeroDivisionError: Dividing by zero ex: 10/0
7. IOError/OSError: OS or) file operation failed.

ex:
```
with open('data.txt','w') as f:
    f.write('Hello')
```

Scanned with OKEN Scanner

Exception Handling:

* We can avoid Runtime errors by using exception handling
* The exception raising code we need to write inside the "try" block
* The exception handling code we need to write inside the "except" block.
* try and except both are keywords in python.

Ex: try:
    # Code block where exception can occour.
except ExceptionType:
    # Handle the exception.

Zero division error:

Ex: try:
    a = 10/0
    if a%2==0:
        print('Even number')
    else:
        print('Odd number')
except Exception as e:
    print('We can't divide with zero')
    print(e)
    print('I'm outside the except block')

O/P: 'We can't divide with zero'
'division by zero'
I'm outside the except block

Value error :

```
try :
    int ('Naveen')
except Exception as e :
        print ('Value error caught :', e)
        print ("I'm outside the except block")
```

o/p: Value error caught : Invalid literal for int()...

Type error :

```
try :
    a='2'+2
except Exception as e :
        print ("Type error caught :", e)
```

o/p:

Type error caught : can only concatenate str (not 'int') to str

Index error :

```
mylist = [1,2,3,10,50,'Naveen','pen']
try :
    print(mylist[50])
except Exception as e :
    print (e)
print('program continous after handling the exception')
```

o/p:-

list of index out of range.
Program continous after handling the exception.

Scanned with OKEN Scanner

Key error:

```
try:
    mydict = {'a':1, 'b':2, 'c':3}
    print(mydict['Naveen'])
except Exception as e:
    print('Keyerror Caught:', e)
print('I'm out of except block')
```

o/p:

Keyerror Caught: 'Naveen'.

Multiple exception Blocks:

```
try:
    x = int(input('Enter the numerical value:'))
    y = 10/x
    print(x, y)
exception ZeroDivisionError:
        print('Division by zero')
exception ValueError:
        print('Invalid input! please enter a
              numerical value')
```

o/p: If x=2 => o/p: 5

x=0 => o/p: Division by zero

x='N' => o/p: 'Invalid input! please .. Value

finally:

* finally is a keyword in python.
* We can use finally keyword for cleanup activities.
* Speciality of finally keyword is, it will executed always irrespective of exception raised or not, exception handled or not
* The main purpose of finally keyword is for closing the files/ database connection.

Raise:

* Raise is a keyword in python.
* By using this we can create our own exception.
* By using raise keyword we can indirectly throwing an object as an exception.

with:

* with is a keyword in python
* The purpose of with keyword is cleanup tasks.
* Generally when we open a file or database connection explicitely we need to close the file or database connection.
* The main advantage of "with" keyword where when we open file with 'with' keyword it will take care of closing the file/ database connection automatically even if an error occurs inside the block

Real time Example:

```
def log_message(message, file_path):
    try:
        with open(file_path, 'a') as file:
            file.write(message)
            print('File opened & message added')
    except FileNotFoundError:
        print('File is not there at given location')
log_message('Hello Naresu', '/content/log.txt')
```

o/p: File is not there at given location.

File Handling:

* File Handling means reading data from files, writing data into files and updating existing files using Python

Ex:
1. Reading employee details from .txt file
2. Saving machine learning model results to a file

```
# code    file = open('example.txt','r')
```

File opening modes:

'r' → Read → Reads an existing file
'w' → Write → Creates or Overwrites a file
'a' → append → Adds data at the end
'r+' → Read + write → Reads and modifies an existing file.
'w+' → write + Read → Creates or clears a file then allows reading
'a+' → Append + Read → Reads and appends to a file.

Example: example.txt

```
Hello Python
Welcome to the file handling
```

1. read() →
```
with open('example.txt', r) as file:
    print(file.read())
```
Reads the entire file.

O/p:
```
Hello Python
Welcome to the file handling
```

Scanned with OKEN Scanner

2. readline()

reads one line.

```
with open('example.txt', r) as file:
    print(file.readline())
    print(file.readline())
```

O/p:

Hello python → For first readline.
Welcome to file handling → For second readline

3. readlines()

Returns all lines as a list

```
with open('example.txt', r) as file:
    print(file.readlines())
```

O/p:

```
["Hello python \n"
 "Welcome to file handling \n"]
```

4. read line by line.

```
with open('example.txt', r) as file:
    for line in file:
        print(line, end=" ")
```

* This is useful for processing large files without loading all thier contents into memory at once.

5. Writing into a file

```
with open('example.txt', w) as file:
    file.write('Monty Python flying circus')
```

* If file does not exist, python creates it, if already exists, its previous content is overwritten.

To write multiple lines.

```
with open('example.txt', w) as file:
    file.write('Hello \n')
    file.write('Naveen \n')
```

* \n moves text to next line

6 File pointer (.seek) :

* The file pointer indicates the current position from which reading or writing takes place.
* The seek() method moves the pointer.

Ex :

```
with open('example.txt',r) as file :
print (file.read(5)) # reads the first five charac
 file.seek(0) # moves the pointer back to the begini
 print (file.read()) # then reads the file from
                       that position.
```

7. Read and write together :

→ r+ mode

* Reads and writes an existing file without automatically clearing it.

Ex :

```
with open('example.txt', 'r+') as file:
    content = file.read()
    print(content)
    file.seek(0)
    file.write('Hello Python')
```

→ w+ mode.

* Creates a file if needed, or clears an existing file. Allows both writing and reading.

Ex :

```
with open('example.txt', "w+") as file:
    file.write ("New content")
    file.seek(0)
    print (file.read())
```

Scanned with OKEN Scanner

→ a+ mode

Allows both reading and appending. The file is created if does not exist

Ex: with open('example.txt', 'a+') as file:
    file.write("\n Another line")
    file.seek(0)
    print(file.read())

→ Character encoding

with open('example.txt', 'r', encoding='UTF-8') as file:
    print(file.read())

* UTF-8 supports texts in many languages and is good default for text lines when you want to encoding explicitly

Quick revision sheet:

open() → opens a file.
read() → Reads all remaining content.
readline() → Reads one line.
readlines() → Returns lines as list
write() → write a string
seek(0) → Moves to the begining
close() → Closes the file.
with → Handles closes automatically.
try-except → Handles file errors.
encoding "UTF-8" → Specifies text encoding.

Scanned with OKEN Scanner